CS-351: Complier Construction

(Spring 2026)

Lecture 3

Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: https://adrashid.github.io/

Recap

- Interpreter Vs Compiler
- Development of Compilers
- Structure of Compilers

Outline

- Structure of Compilers
- Types of Compilers

Lecture's Content

Compiler: Principles, Techniques and Tools (2nd Edition)

by

Alfred V. Aho et al.

Section 1.1 (Page 2 onwards)

Structure of Compiler

- Lexical Analysis
- Parsing
- Semantic Analysis
- Optimization
- Code Generation

Lexical Analysis

- Lexical Analysis divides program text into words or tokens (Converts characters into tokens)
  - Token: Smallest meaningful unit
  - Example:

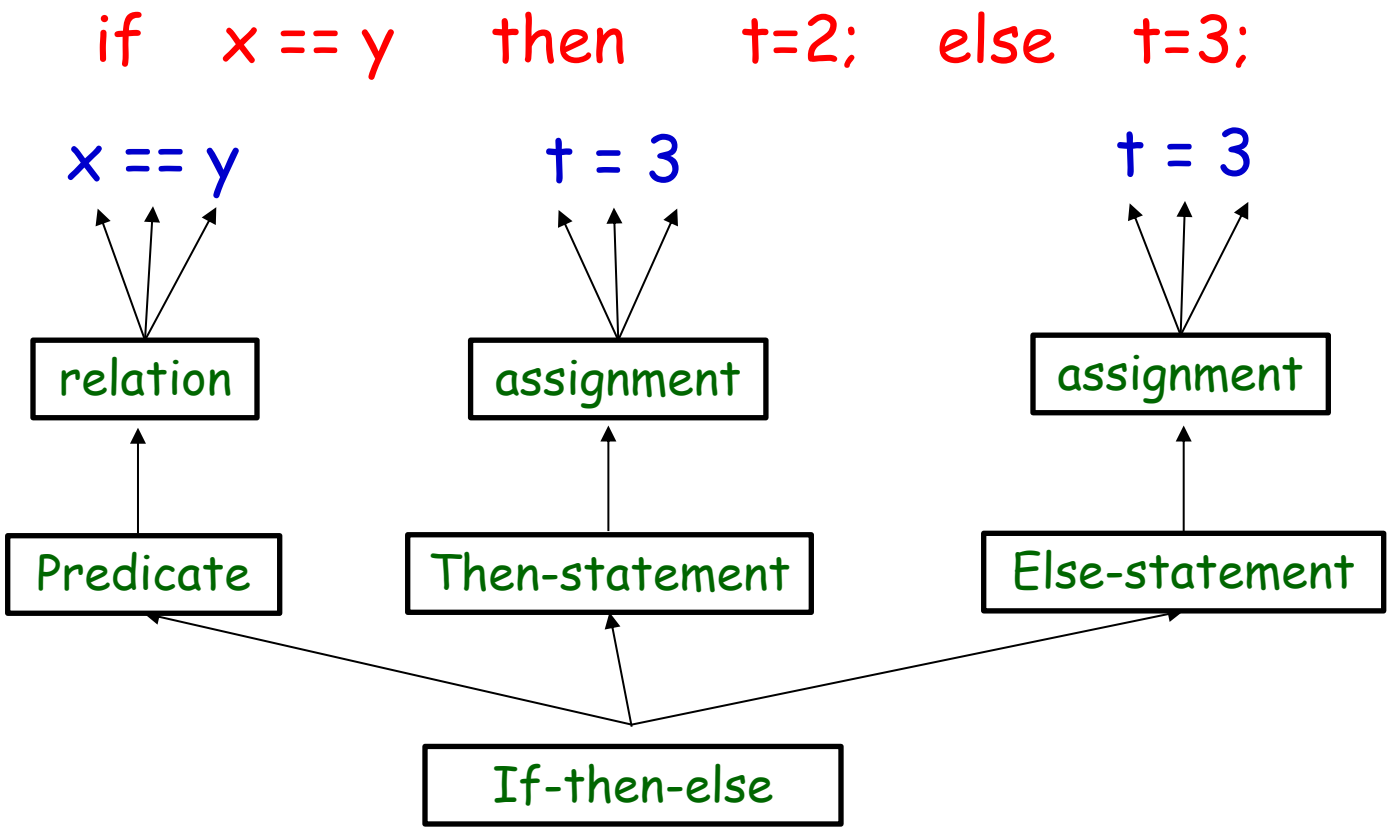
if x == y then t=2; else t=3;

<u>if</u> x <u>==</u> y <u>then</u> t<u>=</u>2; else t=3;

Some tokens/words

Parsing

- Check the grammatical structure or syntax



Semantic Analysis

- Semantic Analysis is a process in compiler construction, after parsing (it ensures the **meaning**)
  - It usually includes type checking
  - It makes sure, a variable is declared before use

- Example:

  int + float should be corrected to:

  - float + float (implicit type checking)

    OR

  - int + int (explicit type checking)

Optimization

A Compiler uses optimization on the source program so that:

- It executes more rapidly
- It occupies less memory
- Draw less power

- Example 1 (Mathematical):

  Simplify all the formulas, i.e.,

  - Instead of $z = \cos^2(x) + 2 + x + \sin^2(x)$;
  - Use $z = x + 3$;

Optimization

- Example 2 (Loop Optimization):

  Simplify all the formulas, i.e.,

```
int i, sum = 0;
for (i = 1; i <= N; ++i) {
   sum += i;
}
printf("sum: %d\n", sum);
```

//Original Code

```
int sum = N * (1 + N) / 2;
printf("sum: %d\n", sum);
```

//Optimized

Code Generation

Code generation is the process by which a compiler's code generator converts some intermediate representation (IR) of source code into a form (e.g., machine code) that can be readily executed by a machine

Code Generation

- Generated code should have minimum the following properties:
  - It should carry the exact meaning of the source code
  - It should be efficient in terms of CPU usage and memory management

Time Consumed on Compiler Phases

In 1960s:

Lexical Analysis	Parsing	Semantic Analysis	Code Optimization	Code Generation

In 2010s and afterwards:

Lexical Analysis	Parsing	Semantic Analysis	Code Optimization	Code Generation

Huge time spent on code optimization these days

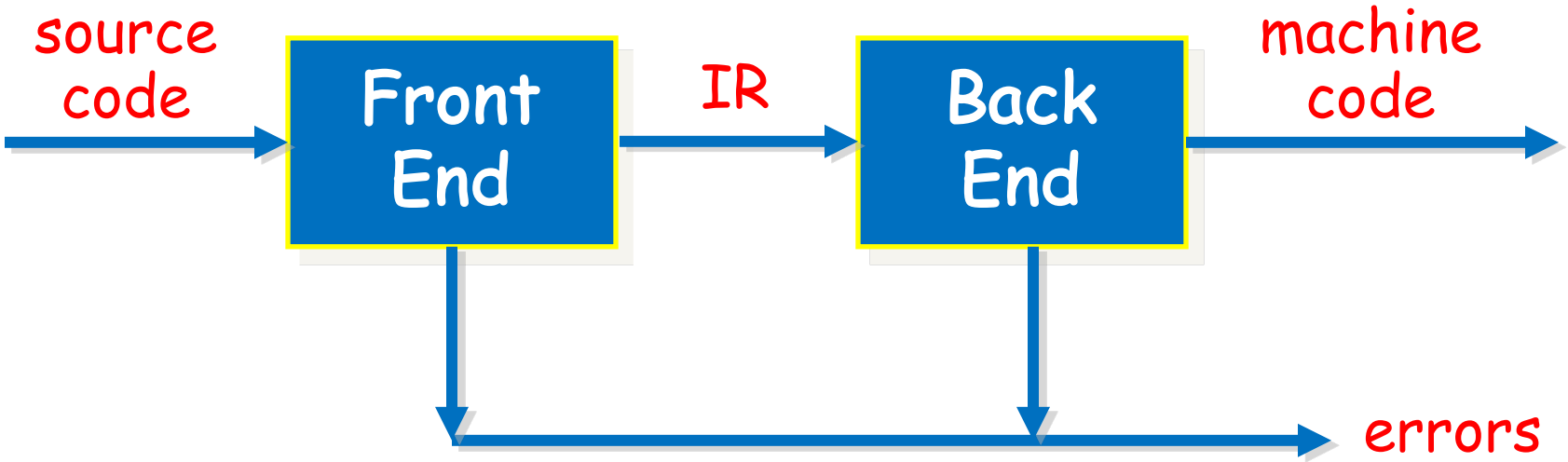
Type of Compilers

Type of Compilers

Type of Compilers

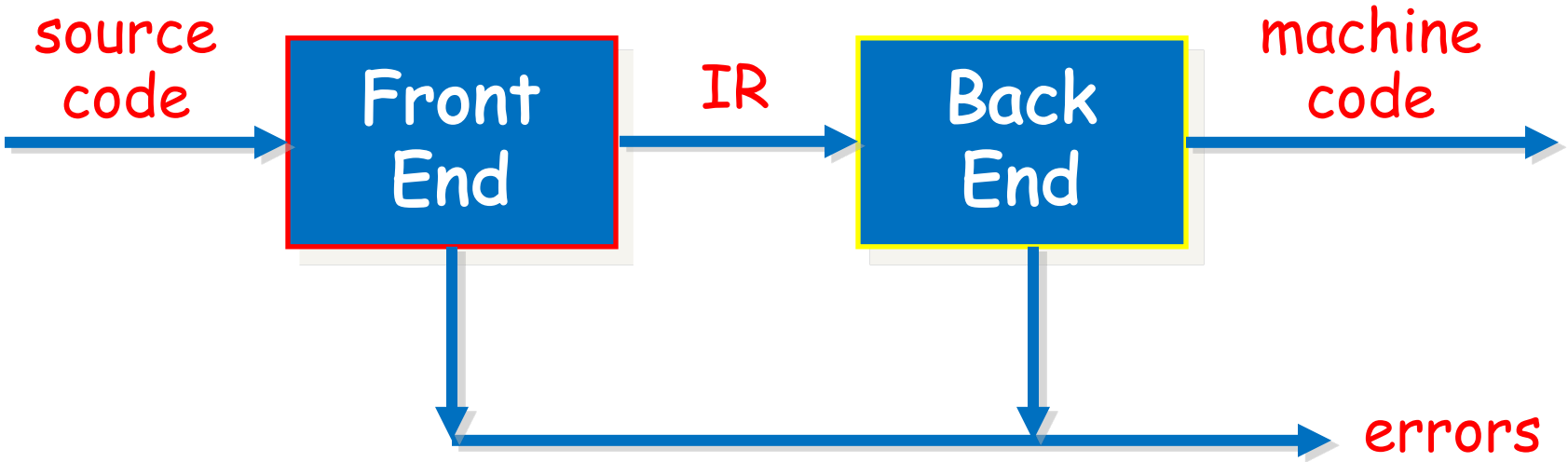
- Two Pass Compiler
- Three Pass Compiler

Two Pass Compiler



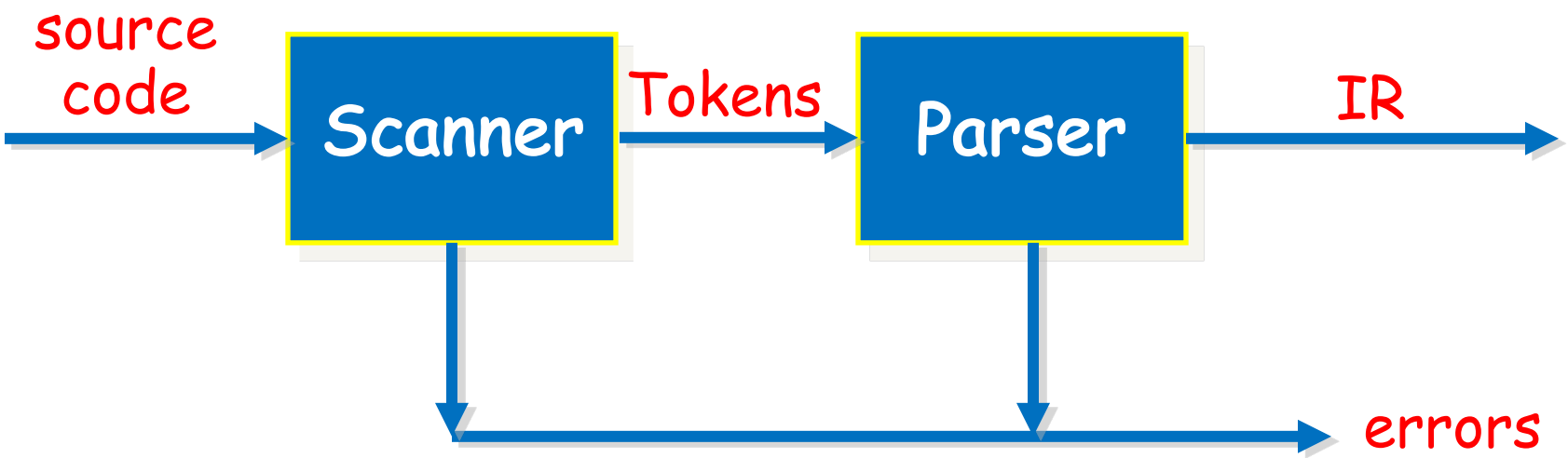
- Use an Intermediate Representation (IR)
- Front End maps legal source code into IR
- Back End maps IR to target machine code
- Admits multiple front ends and multiple passes

Two Pass Compiler



- Recognizes legal (and illegal) programs
- Report errors in a useful way
- Produce IR and preliminary storage map

The Front End



Modules

- Scanner (Lexical Analyzer): Maps character stream into words – basic unit of syntax
- Parser: Recognizes context-free syntax and reports errors

Scanner (Lexical Analyzer)

- Maps character stream into words – basic unit of syntax
- Produces pairs
  - a word and
  - its part of speech
- Example

  x = x + y

  becomes

  <id,x>
  <assign,=>
  <id,x>
  <op,+>
  <id,y>

<id,x>

token type

attribute value

We call the pair:
<token type, value> a token
Typical tokens: number, identifier, +, -, new, while, if

Thank you!